

Локализация Python

Для локализации строковых констант в исходном коде Python необходимо использовать функцию `rk()`. Она принимает следующие параметры:

1. Строка для перевода - строка, которую механизм интернационализации будет искать в словаре.
2. Контекст - строка, описывающая [контекст перевода](#) для переводчика. Является необязательным параметром.
3. Число - число, которое задает количество объектов для выбора склонения.



Импортирование функции `rk()`

Функция `rk()` подключается вместе с библиотекой `sbis`. Подключать какие-то другие библиотеки не требуется.

Локализация строковых констант

```
1 from sbis import rk
2 ...
3 LogMsg(rk('Функция checkout устарела', 'Мой функционал'))
```

или

```
1 import sbis
2 ...
3 LogMsg(sbis.rk('Функция checkout устарела', 'Мой функционал'))
```

Не допускается создание статических констант, обернутых в `rk`.

```
1 # Неправильно
2
3 class A:
4     # статическая переменная класса, которая создается один раз для
    всех экземпляров классов
5     message = sbis.rk("мое любимое сообщение")
6     def get_message():
7         # здесь нет вызова rk, а значит, что сообщение не будет
    переведено
8         return A.message

1 # Правильно
2
3 class A:
4     message = "мое любимое сообщение"
5     def get_message():
6         # перевод будет подставляться при каждом вызове метода
7         return sbis.rk(A.message)
```

Конкатенация строк

В исходном коде не допускается склеивание переводимых строк:

```
1 # Неправильно
2
```

```

3 from sbis import rk
4 ...
5 raise Error(rk('Ошибка') + ': ' + message + ' ' + rk('с кодом') + ' '
+ str(code))

```

Порядок слов в предложении зависит от языка перевода, склейка препятствует правильному переводу.

Вместо склейки необходимо использовать подстановки (плейсхолдеры):

```

1 # Правильно
2 from sbis import rk
3 ...
4 raise Error(rk('Ошибка: "{0}" с кодом {1}').format(message, code))

```

Допускается использование именованных подстановок:

```

1 # Правильно
2 from sbis import rk
3 ...
4 raise Error(rk('Случилась ошибка: "{message}" с кодом
{code}').format(message=some_message, code=some_code))

```



Подстановку необходимо производить после вызова функции `rk`, поэтому подстановки должны быть во всех переводах ключа.

Пробелы, управляющие символы в начале и в конце локализуемых строк

Не допускается использование [управляющих символов](#) (`\t` — горизонтальная табуляция; `\n` — перевод строки; `\r` — возврат каретки) в начале и в конце переводов.

Ключи с невидимыми символами на концах строки сильно затрудняют перевод. Например, строки **Сохранить**, `\nСохранить` и `Сохранить\r` придётся переводить отдельно.

1. Локализуемая строка не должна содержать символы пробелов в начале или в конце:

```

1 from sbis import rk
2 ...
3 # Неправильно
4 rk(' Текст ')
1 from sbis import rk
2 ...
3 # Правильно
4 rk('Текст')

```

2. Локализуемая строка не должна содержать символы перевода строки.

```

1 from sbis import rk
2 ...
3 # Неправильно
4 rk('\nТекст')
1 from sbis import rk
2 ...
3 # Правильно
4 "\n" + rk('Текст')

```

Знаки препинания

Знаки препинания включают в локализуемую строку:

```

1 from sbis import rk
2 ...
3 # Неправильно
4 rk('Ошибка') + '!'
5 rk('Предупреждение') + ':'
6 rk('Удалить') + '?'

1 from sbis import rk
2 ...
3 # Правильно
4 rk('Ошибка!')
5 rk('Предупреждение:')
6 rk('Удалить?')

```

Склонение слова в зависимости от числа

При склонении слова в зависимости от числа предполагается, что ключ будет содержать предполагаемые окончания переводов:

```

1 запись(-и, -ей)

```

К исходной строке добавляются скобки с вариантами окончания слова. Такой вид строки нужен только для того, чтобы выделить её в [словаре переводов](#) среди остальных строк. Добавленные в скобках окончания никак не повлияют на перевод или форму слова при локализации. Если на сервисе отключена локализация, то строка будет выводиться в исходном виде.

Сигнатура функции rk():

```

1 # ЛОКАЛИЗАЦИЯ + КОНТЕКСТ + СКЛОНЕНИЕ.
2 # key: str – ключ словаря локализации.
3 # context: str – контекст перевода.
4 # number: int – число, соответствующее склонению.
5 rk(key, context, number)
6
7 # ЛОКАЛИЗАЦИЯ + СКЛОНЕНИЕ.
8 rk(key, number)

```

Примеры:

```

1 # ЛОКАЛИЗАЦИЯ + СКЛОНЕНИЕ
2 # Вернёт "запись" или "record"
3 rk( "запись(-и,-ей)", 1 )
4 # Вернёт "записи" или "records".
5 rk( "запись(-и,-ей)", 2 )
6 # Вернёт "записей" или "records"
7 rk( "запись(-и,-ей)", 5 )
8 # Вернёт "записи" или "records".
9 rk( "запись(-и,-ей)", 1.5 )
10 # ЛОКАЛИЗАЦИЯ + КОНТЕКСТ + СКЛОНЕНИЕ
11 # Вернёт "день" или "day".
12 rk( "день(-я,-ей)", "контекст", 1 )
13 # Вернёт "дня" или "days".
14 rk( "день(-я,-ей)", "контекст", 2 )
15 # Вернёт "дней" или "days".
16 rk( "день(-я,-ей)", "контекст", 5 )
17 # Вернёт "дня" или "days".

```

```
18 rk( "день(-я, -ей)", "контекст", 1.5 )
```

Число в конце функции означает количество единиц объектов. От этого числа зависит выбор формы перевода. Количество склонений для языков разное. Для русского языка (**ru***) должно быть четыре формы перевода (1 объект, 2-4 объекта, 5+ объектов, дробное количество объектов), а для английского (**en***) — две формы (1 object, 2+/fractional quantity of objects).

В словарь перевода добавляем такие строки:

```
1 /* ru.json */
2 {
3   "plural#запись(-и, -ей)": "запись|записи|записей|записи",
4   "контекст@@plural#день(-я, -ей)": "день|дня|дней|дня"
5 }

1 /* en.json */
2 {
3   "plural#запись(-и, -ей)": "record|records",
4   "контекст@@plural#день(-я, -ей)": "day|days"
5 }
```